// T-OLIMPIANS · ARKUSZ REFERENCYJNY

Metryki i obrót płaszczyzny

Euklidesowa, Manhattan, Czebyszewa, rzeka i centrum — plus obracanie układu współrzędnych. Geometria na konkursy informatyczne — OIJ, OI, IOI.

metryka euklidesowa | Manhattan (L1) | Czebyszewa (L∞) | obrót o 45° | obrót o kąt β | wzory na sumę kątów | M_PI / radiany | metryka rzeka | metryka centrum | superźródło

01 Trzy metryki na płaszczyźnie

Metryka to sposób mierzenia odległości między punktami P1(x1, y1) i P2(x2, y2). Zadanie prawie nigdy nie mówi wprost „to metryka Czebyszewa" — poznajesz ją po tym, **jak wolno się poruszać**.

Metryka	Wzór	Ruch
Euklidesowa (L2)	`d = sqrt((x1 - x2)^2 + (y1 - y2)^2)`	w linii prostej
Miejska / Manhattan (L1)	`d = |x1 - x2| + |y1 - y2|`	po kratach
Czebyszewa (L∞)	`d = max(|x1 - x2|, |y1 - y2|)`	jak król w szachach

Jak je rozpoznać: Manhattan — chodzisz po ulicach miasta, wolno tylko góra / dół / lewo / prawo, jeden krok = 1.
Czebyszewa — wolno też na ukos, a krok na ukos kosztuje tyle samo co prosto (król szachowy, ruch 8-kierunkowy).
Euklidesowa — lecisz na wprost, po przekątnej płacisz sqrt(2).

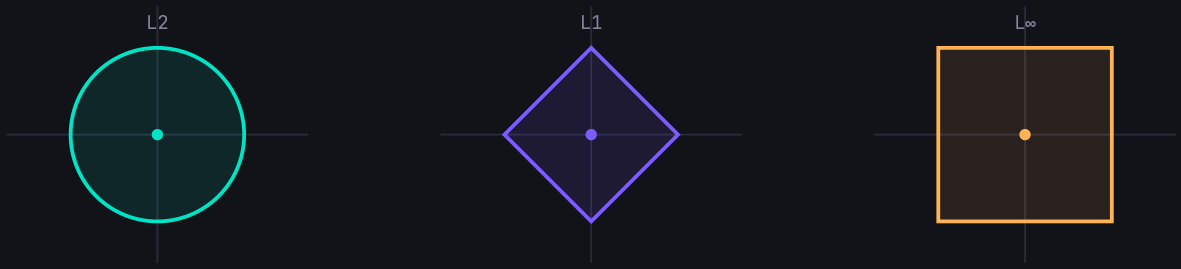
```
// wszystkie trzy naraz — dx, dy zawsze w long long
long long dx = abs(x1 - x2), dy = abs(y1 - y2);

long long manhattan = dx + dy;
long long czebyszew = max(dx, dy);
long long euklides2 = dx * dx + dy * dy;   // kwadrat — bez sqrt
```

Pułapka konkursowa: w euklidesowej **nie licz pierwiastka, jeśli nie musisz** — do porównywania odległości wystarczy kwadrat, a double gubi precyzję. Kwadraty licz w long long: dx*dx przepełnia int już przy dx ≈ 46341.

02 Kule jednostkowe — dlaczego akurat te kształty

„Kula jednostkowa" to zbiór wszystkich punktów odległych o dokładnie 1 od środka. Kształt kuli to wizytówka metryki — i podpowiedź, dlaczego Manhattan i Czebyszewa da się zamienić jedną na drugą.



euklidesowa — koło | Manhattan — romb | Czebyszewa — kwadrat

Romb i kwadrat to **ten sam kształt obrócony o 45°** — stąd bierze się cała sztuczka z sekcji 03.

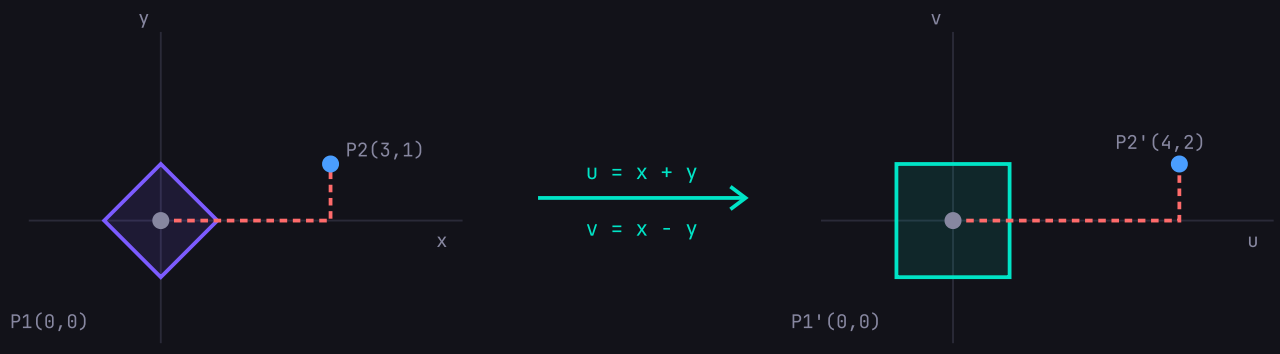
Skąd te kształty:
L1: |x| + |y| = 1 to cztery odcinki łączące (1,0), (0,1), (-1,0), (0,-1) → romb.
L∞: max(|x|, |y|) = 1 to cztery boki x = ±1 i y = ±1 → kwadrat o boku 2.

03 Manhattan ↔ Czebyszewa — obrót o 45°

Najważniejsza sztuczka z całego tematu. Metryka Manhattan ma sumę dwóch wartości bezwzględnych — ciężko ją optymalizować. Metryka Czebyszewa ma `max` — a maksimum łatwo liczyć po każdej współrzędnej osobno. Dlatego **przerzucamy zadanie do Czebyszewa**.

Manhattan → Czebyszewa $(x, y) \to (x + y, x - y)$

Czebyszewa → Manhattan $(x, y) \to ((x + y) / 2, (x - y) / 2)$



Manhattan: |3| + |1| = 4

Czebyszewa: max(4, 2) = 4

Ten sam wynik, ale po prawej liczony jako **zwykłe maksimum, bez sumowania**.

Dlaczego to działa: po podstawieniu $u = x + y$, $v = x - y$ fioletowy romb (kula L_1) ląduje jako zielony kwadrat (kula L_∞). Romb obrócony o 45° i przeskalowany to dokładnie kwadrat, więc **przekształcenie jest bezstratne** — nie zniekształca odległości, tylko zmienia układ współrzędnych tak, żeby „po skosie” (Manhattan) stało się „wprost” (Czebyszewa).

Sztandarowe zastosowanie: **najdalsza para punktów w metryce Manhattan**. Wprost to O(n^2) par — po przejściu do Czebyszewa wystarczy jedno przejście po tablicy.

```
long long mnU = LLONG_MAX, mxU = LLONG_MIN;
long long mnV = LLONG_MAX, mxV = LLONG_MIN;

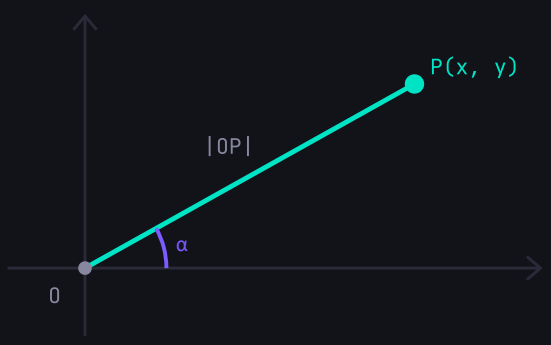
for (int i = 0; i < n; i++) {
    long long u = x[i] + y[i];      // Manhattan -> Czebyszew
    long long v = x[i] - y[i];
    mnU = min(mnU, u);  mxU = max(mxU, u);
    mnV = min(mnV, v);  mxV = max(mxV, v);
}

cout << max(mxU - mnU, mxV - mnV) << "\n";   // O(n)
```

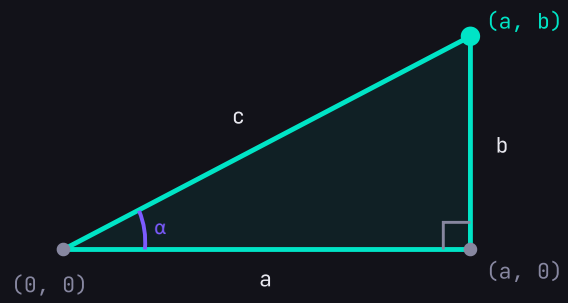
Pułapka konkursowa: powrót dzieli przez 2, więc na współrzędnych całkowitych **albo w ogóle nie wracaj** (licz odpowiedź już w Czebyszewie), albo pilnuj parzystości. Uwaga na zakres: $u = x + y$ podwaja wartości — przy $x, y \le 10^9$ trzymaj u, v w `long long`.

04 Obracanie płaszczyzny — zapis biegunowy

Mamy punkt P(x, y). Żeby go obrócić, najpierw zapisujemy go inaczej: przez **długość** odcinka |OP| i **kąt** α, jaki tworzy z osią X. Obrót zmienia wtedy tylko kąt.



Każdy punkt to **długość + kąt**.



Dowód — zwykły **trójkąt prostokątny**.

każdy punkt można zapisać jako

P(|OP| * cos α, |OP| * sin α)

```
// dowód — z definicji funkcji trygonometrycznych w trójkącie prostokątnym:
cos α = a / c    sin α = b / c    |OP| = c
⇒  |OP| * cos α = c * (a / c) = a    // czyli x
⇒  |OP| * sin α = c * (b / c) = b    // czyli y
```

Kluczowa obserwacja: podczas obrotu |OP| się **nie zmienia** — zmienia się jedynie kąt. Czyli punkt obrócony o β stopni w stronę przeciwną do ruchu wskazówek zegara to:
P'(|OP| * cos(α + β), |OP| * sin(α + β))

Wzory na sumę i różnicę kątów (są też w karcie wzorów maturalnych)

```
sin(α + β) = sin α cos β + cos α sin β
sin(α - β) = sin α cos β - cos α sin β
cos(α + β) = cos α cos β - sin α sin β
cos(α - β) = cos α cos β + sin α sin β
```

```
// zapis biegunowy w kodzie — tam i z powrotem
double r    = hypot(x, y);      // |OP|
double alfa = atan2(y, x);      // kat alfa, w RADIANACH
double nx   = r * cos(alfa);   // wraca do x
double ny   = r * sin(alfa);   // wraca do y
```

Uwaga: |OP| i α to tylko **rusztowanie dowodu** — w gotowym wzorze z sekcji 05 żadne z nich nie występuje. Do samego obrotu nigdy nie musisz ich liczyć, więc nie ma tam ani pierwiastka, ani atan2.

05 Wzór na obrót — rozpisanie

Podstawiamy wzory na sumę kątów i zwijamy z powrotem do x i y. Cały trik polega na tym, że |OP| cos α to po prostu x, a |OP| sin α to y.

```
// P(x, y) ── obrót o β → P'(x', y')
=  P'( |OP| cos(α + β),   |OP| sin(α + β) ) =
=  P'( |OP|(cos α cos β - sin α sin β),   |OP|(sin α cos β + cos α sin β) ) =
=  P'( |OP| cos α cos β - |OP| sin α sin β,   |OP| sin α cos β + |OP| cos α sin β ) =
=  P'( x cos β - y sin β,   y cos β + x sin β )
```

czyli — obrót o kąt β przeciwnie do wskazówek zegara

$$x' = x * \cos\beta - y * \sin\beta$$

$$y' = y * \cos\beta + x * \sin\beta$$

Kąt β	Co robi	Nowe (x', y')
90°	ćwierć obrotu w lewo	(-y, x)
180°	odbicie przez środek	(-x, -y)
270°	ćwierć obrotu w prawo	(y, -x)
45°	skos → prosto (Manhattan → Czebyszewa)	((x - y)/sqrt(2), (x + y)/sqrt(2))

Skąd obrót o 45° w sekcji 03: podstaw β = 45°, wtedy cos β = sin β = sqrt(2)/2, więc x' = (x - y)*sqrt(2)/2 i y' = (x + y)*sqrt(2)/2. Odrzucamy wspólny czynnik sqrt(2)/2 — skalowanie nie psuje porównywania odległości — i zostaje czyste (x - y, x + y) **na liczbach całkowitych**. Kolejność współrzędnych nie ma znaczenia, więc zwykle zapisuje się to jako (x + y, x - y).

Pułapka konkursowa: dla obrotów o wielokrotność 90° **nie wołaj sin/cos** — użyj gotowych wzorów z tabeli. cos(M_PI/2) to nie idealne 0, tylko 6.1e-17, i po kilku obrotach współrzędne całkowite rozjeżdżają się na ułamki.

```
// przykład: obróć P(3, 0) o 90° przeciwnie do wskazówek zegara
cos 90° = 0    sin 90° = 1
⇒  x' = 3 * 0 - 0 * 1 = 0
⇒  y' = 0 * 0 + 3 * 1 = 3    // P'(0, 3) — punkt z osi X wskoczył na oś Y
```

Obrót w drugą stronę (zgodnie ze wskazówkami zegara) to podstawienie -β. Cosinus się nie zmienia, sinus zmienia znak, więc wystarczy przestawić dwa minusy:
x' = x cos β + y sin β   y' = y cos β - x sin β

06 Kod — obrót punktu w C++

Obrót wokół początku układu (0, 0) — przepisany wprost wzór z sekcji 05.

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Point { double x, y; };

// obrot wokol punktu origin (0,0) o kat theta
Point obrot(Point p, double theta) {
    double c = cos(theta);
    double s = sin(theta);

    Point result;
    result.x = p.x * c - p.y * s;
    result.y = p.x * s + p.y * c;
    return result;
}
```

A teraz obrót wokół **dowolnego** punktu center — trzy kroki, każdy w osobnej linijce.

```cpp
// obrot wokol dowolnego punktu center, krok po kroku
Point obrotWokol(Point p, Point center, double theta) {
    // krok 1: przesun punkt tak, jakby center byl w (0,0)
    Point shifted;
    shifted.x = p.x - center.x;
    shifted.y = p.y - center.y;

    // krok 2: obroc wokol (0,0), uzywajac funkcji ktora juz mamy
    Point rotated = obrot(shifted, theta);

    // krok 3: przesun z powrotem, dodajac center
    Point result;
    result.x = rotated.x + center.x;
    result.y = rotated.y + center.y;
    return result;
}

int main() {
    Point P = {5, 4}, C = {2, 2};
    Point P2 = obrotWokol(P, C, M_PI / 2);   // 90 stopni wokol C

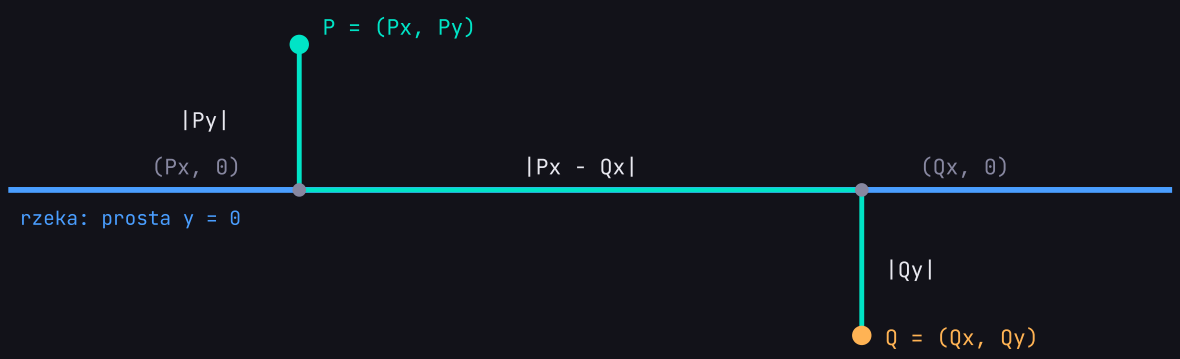
    cout << P2.x << " " << P2.y << "\n";
}
```

Obracanie dookoła dowolnego punktu — idea:
1) przesuwamy nasz punkt tak, żeby center (punkt, dookoła którego obracamy) wylądował w (0, 0)
2) obracamy normalnie, zwykłym wzorem
3) przesuwamy z powrotem tam, gdzie było

Pułapka konkursowa: funkcje cos i sin oczekują **radianów, nie stopni** — zamiana to rad = stopnie * M_PI / 180. Stała M_PI to π, siedzi w <cmath>, więc bits/stdc++.h ją wciąga.
Nie nazywaj tej funkcji rotate — pod using namespace std; koliduje z std::rotate z <algorithm> i potrafi dać nieczytelny błąd kompilacji.

07 Metryka rzeka

Wyobraź sobie rzekę wzdłuż prostej $y = 0$. Poruszać się wolno tylko **pionowo** (prostopadle do rzeki, żeby do niej dojść) oraz **wzdłuż rzeki**. Żeby przejść z P do Q, trzeba więc zejść do rzeki, przepłynąć kawałek i wyjść.



trasa: $P \to (P_x, 0) \to (Q_x, 0) \to Q$

Do rzeki **prostopadle**, potem **wzdłuż** rzeki, potem znowu prostopadle.

gdy $P_x = Q_x$ (oba punkty na jednej pionowej prostej)

$$d(P, Q) = |P_y - Q_y|$$

w przeciwnym razie

$$d(P, Q) = |P_y| + |P_x - Q_x| + |Q_y|$$

```cpp
long long rzeka(long long px, long long py, long long qx, long long qy) {
    if (px == qx) return llabs(py - qy);       // prosto w pionie
    return llabs(py) + llabs(px - qx) + llabs(qy);
}
```

```
// przykład: P = (2, 3), Q = (7, -4), rzeka na prostej y = 0
Px ≠ Qx ⇒  d = |3| + |2 - 7| + |-4| = 3 + 5 + 4 = 12
```

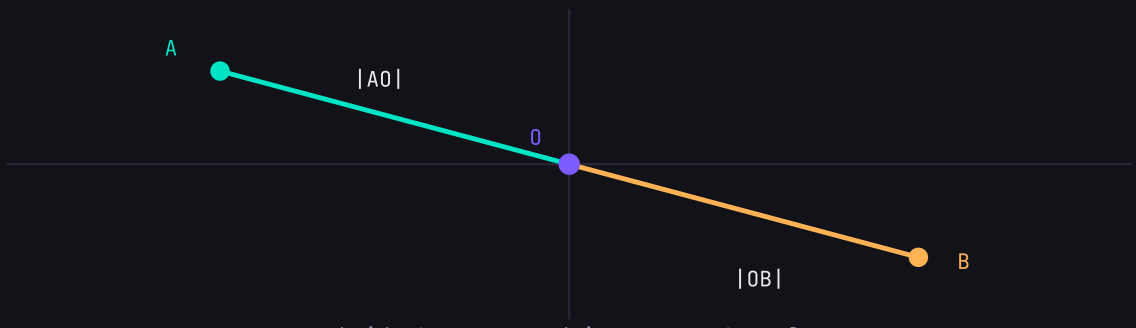
Pułapka konkursowa: nie zapomnij o pierwszym przypadku. Gdy $P_x = Q_x$, idzie się **prosto w pionie** i rzeki się nie dotyka — ogólny wzór dałby tu za dużo, np. dla $P = (0, 5)$, $Q = (0, 3)$ poprawnie jest 2, a nie 8.

Rzeka pod kątem: rzeka nie musi być linią poziomą ani pionową — może iść na ukos. Wtedy **obracamy całą płaszczyznę** (sekcje 04-06) tak, żeby ta linia stała się poziomą prostą $y = 0$, i dalej liczymy zwykłym wzorem. Kąt obrotu bierzemy z atan2 kierunku rzeki.

Gdzie to spotkasz — „most przez rzekę”: poruszasz się po siatce miast, a most / tunel / autostrada pozwala tanio zmienić współrzędną x tylko wzdłuż jednej ustalonej linii (rzeki, torów, głównej ulicy). To dokładnie ta sama struktura: **swobodny ruch pionowy wszędzie, ale zmiana poziomej pozycji dopiero po wejściu na „autostradę”.**

08 Metryka centrum (studnia)

Jest jeden wyróżniony punkt O — centrum. Żeby przejść z A do B, **trzeba przejść przez centrum**: najpierw z A do O, potem z O do B. Jedyny wyjątek to A = B, czyli zero kroków.



każda trasa prowadzi przez centrum O

Gwiazda: wszystko łączy się **tylko przez jeden węzeł**.

gdy A = B $d(A, B) = 0$ w przeciwnym razie $d(A, B) = |AO| + |OB|$

To dokładnie model, który stoi za klasycznym trikiem: wirtualny węzeł w BFS / Dijkstrze.

Wcielenie w zadaniu	Na czym polega
Zadania typu „hub”	n miast, każde łączy się bezpośrednio tylko ze stolicą (lotnisko przesiadkowe, serwer centralny) — koszt to `koszt(A→stolica) + koszt(stolica→B)`. Dosłownie metryka centrum.
Superźródło	Liczysz „odległość do najbliższego z k punktów” — dodajesz wirtualny węzeł O połączony krawędziami wagi 0 ze wszystkimi k punktami. Odwrotność centrum: wiele źródeł zbiega się w jednym punkcie. Pożary, woda, zarażenie w gridzie.
Drzewo + LCA	Odległość w drzewie to `dist(A→LCA) + dist(LCA→B)` — strukturalnie to samo, tylko „centrum” jest inne dla każdej pary (ich LCA), a nie stałe.
Teleport / portal	Jeden portal w gridzie: `koszt = dist(A→portal) + dist(portal→B)`, chyba że droga wprost jest krótsza. Odpowiedź to `min(bezpośrednio, |AO| + |OB|)`.

Trik z superźródłem w praktyce: zamiast puszczać BFS k razy z każdego źródła (O(k·n)), wrzuć na start **wszystkie źródła naraz** — jedno przejście, O(n), i od razu masz odległość do najbliższego źródła dla każdego pola.

```
// superzrodlo: wszystkie zrodla laduja w kolejce z dystansem 0
queue<pair<int, int>> q;
for (auto [r, c] : zrodla) {
    dist[r][c] = 0;
    q.push({r, c});
}
// dalej zwykly BFS — dist[][] to odleglosc do NAJBLIZSZEGO zrodla
```

09 Tabela podsumowująca

Metryka	Wzór	Kula jednostkowa	Po czym poznać
euklidesowa	sqrt(dx^2 + dy^2)	koło	lot w linii prostej
Manhattan	|dx| + |dy|	romb	ruch po kratach, 4 kierunki
Czebyszewa	max(|dx|, |dy|)	kwadrat	8 kierunków, skos za darmo
rzeka	|Py| + |dx| + |Qy|	—	jedna uprzywilejowana linia
centrum	|AO| + |OB|	gwiazda	wszystko przez jeden węzeł

Pułapka konkursowa: dx i dy to skróty na x1 - x2 i y1 - y2. Przy współrzędnych do 10^9 **wszystko licz w long long** — dx*dx przepełnia int już przy dx ≈ 46341.

10 Co zrobić, gdy widzisz...

suma |dx| + |dy| do zoptymalizowania
obrót o 45° — przejdź na (x+y, x−y)

najdalsza para punktów (Manhattan)
max(u) - min(u) i max(v) - min(v)

ruch 8-kierunkowy, skos za 1
metryka Czebyszewa — max(|dx|, |dy|)

prosta / rzeka pod kątem
obróć płaszczyznę — niech to będzie y = 0

most / autostrada / tory
metryka rzeka — sekcja 07

stolica, hub, portal
metryka centrum — wirtualny węzeł

odległość do najbliższego z k źródeł
superźródło — wszystkie do kolejki naraz

obrót o 90 / 180 / 270 stopni
(-y, x) / (-x, -y) / (y, -x)

porównywanie odległości euklidesowych
zostaw kwadraty — bez sqrt, bez double

odległość dwóch wierzchołków w drzewie
LCA — centrum inne dla każdej pary

11 Materiały i zadania

Rodzaj	Gdzie
Nagranie — Metryka miejska / Manhattan (#26 Olimpiada Zaawansowana)	youtube.com/live/jqXCeDwOV78
Zadanie — CSES	cses.fi/problemset/task/3410
Zadanie — Szkopuł	szkopul.edu.pl/problemset/problem/Nj5KslMblkAfffMZtJPVBjxl

Kolejność nauki: najpierw rozpoznawanie metryki z treści zadania (sekcje 01–02), potem obrót o 45° (03) — to on daje najwięcej punktów. Wyprowadzenie (04–05) warto zrozumieć raz, a potem po prostu pamiętać dwie linijki wzoru.